

Why am I getting a weird error about `promise_type` when I try to write a coroutine? part 2

 devblogs.microsoft.com/oldnewthing/20230322-00

March 22, 2023



Raymond Chen

A customer was having trouble getting their coroutine to compile. They shared this fragment:

```
switch (args.Key())
{
case winrt::VirtualKey::A:
{
    winrt::ContentDialog dlg;
    dlg.Title(winrt::box_value(L"Title of my dialog"));
    dlg.Content(winrt::box_value(L"Text of the dialog"));
    dlg.PrimaryButtonText(L"Click this guy");
    dlg.SecondaryButtonText(L"Not this guy");
    auto result = co_await dlg.ShowAsync();

    switch (result)
    {
        ...
    }
}
break;
}
```

The error that resulted from the `co_await` was

```
error C2039: 'promise_type': is not a member of 'std::coroutine_traits<void,winrt::
Contoso::implementation::AwesomeControl &,winrt::Windows::UI::Xaml::Input::
KeyRoutedEventArgs>'
```

They found [an earlier discussion of this error](#) and reported that the recommended fix did not help. They were already including `winrt/Windows.Foundation.h`, but the error persisted.

Recall that when you `co_await`, the compiler looks for a `coroutine_traits<T, ...>::promise_type`, where `T` is the return type of the function, and the other stuff is rarely of any significance.

The customer skipped over the “How to understand the error message” part and went right to “What did the error message mean in this case?” part, even though the case under study didn’t match their problem.

It’s like listening to a radio car repair call-in show, hearing somebody say, “My car won’t start,” ignoring all of the analysis from the hosts, and skipping to the part where the host says “It sounds like you need new spark plugs.” Your car doesn’t start, so you replace the spark plugs, but it doesn’t help. What you skipped was the back-and-forth between the host and the caller to figure out that the problem was the spark plugs.

Instead, let’s apply the discussion of the problem (rather than its conclusion). The problem is that we couldn’t find `coroutine_traits<void, ...>::promise_type`.

And that’s the crux of the problem.

Nobody has defined what it means to create a coroutine from a function that returns `void`. Including `winrt/Windows.Foundation.h` won’t help. That header file defines how to create a coroutine from a function that returns `IAsyncAction`, `IAsyncOperation`, and the related progress types. It doesn’t define anything about `void`.

From this, we can infer from the code the customer didn’t share that the `co_await` is happening in a function declared as returning `void`. The compiler is complaining that nobody has taught it how to `co_await` in a function that returns `void`.

The C++/WinRT return types that support being used as coroutines are `fire_and_forget` and the various `IAsyncXxx` types, so choose one of those. You probably want `fire_and_forget`.